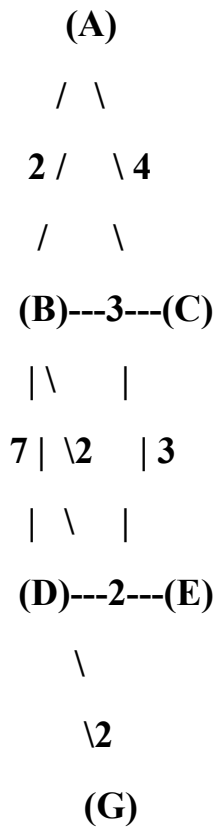


DRY RUN TEST

Question 1: Dry Run of A Search Algorithm (10 Marks)*

Consider the following graph:



Heuristic Values:

Node	$h(n)$
A	7
B	6
C	4
D	3
E	2
G	0

Task

Perform a dry run of the A* Search algorithm from A to G.

For each iteration, show:

- Current Node
- Open List
- Closed List
- $g(n)$
- $h(n)$
- $f(n) = g(n) + h(n)$

Finally,

1. Identify the optimal path.
2. Calculate the total path cost.

CODE :

```
import heapq

graph = {
    'A': [('B', 2), ('C', 4)],
    'B': [('A', 2), ('C', 3), ('D', 7)],
    'C': [('A', 4), ('B', 3), ('E', 3)],
    'D': [('B', 7), ('E', 2)],
    'E': [('C', 3), ('D', 2), ('G', 2)],
    'G': []
}

heuristic = {
    'A': 7,
    'B': 6,
```

```

'C': 4,
'D': 3,
'E': 2,
'G': 0
}

def astar(start, goal):
    pq = []
    heapq.heappush(pq, (heuristic[start], 0, start, [start]))
    visited = set()
    print("\nA* Search Steps\n")
    while pq:
        f, g, node, path = heapq.heappop(pq)
        if node in visited:
            continue
        visited.add(node)
        print("-----")
        print("Current Node :", node)
        print("Path So Far :", " -> ".join(path))
        print("g(n) =", g)
        print("h(n) =", heuristic[node])
        print("f(n) =", f)
        if node == goal:
            print("\nGoal Reached!")
            print("Optimal Path :", " -> ".join(path))
            print("Total Cost :", g)
            return

```

```

print("Neighbours:")
for neighbour, cost in graph[node]:
    if neighbour not in visited:
        new_g = g + cost
        new_f = new_g + heuristic[neighbour]
        print(f'{node} -> {neighbour}')
        print(f'Cost = {cost}')
        print(f'g = {new_g}')
        print(f'h = {heuristic[neighbour]}')
        print(f'f = {new_f}\n')
        heapq.heappush(
            pq,
            (new_f, new_g, neighbour, path + [neighbour])
        )
start = input("Enter Start Node: ").upper()
goal = input("Enter Goal Node: ").upper()
astar(start, goal)

```

OUTPUT :

```
Output
Enter Start Node: A
Enter Goal Node: D

A* Search Steps

-----
Current Node : A
Path So Far : A
g(n) = 0
h(n) = 7
f(n) = 7
Neighbours:
A -> B
Cost = 2
g = 2
h = 6
f = 8

A -> C
```

```
Output

A -> C
Cost = 4
g = 4
h = 4
f = 8

-----
Current Node : B
Path So Far : A -> B
g(n) = 2
h(n) = 6
f(n) = 8
Neighbours:
B -> C
Cost = 3
g = 5
h = 4
f = 9
```

Output

```
B -> D
Cost = 7
g = 9
h = 3
f = 12
```

```
-----
Current Node : C
Path So Far   : A -> C
g(n) = 4
h(n) = 4
f(n) = 8
Neighbours:
C -> E
Cost = 3
g = 7
h = 2
f = 9
```

Output

```
-----
Current Node : E
Path So Far   : A -> C -> E
g(n) = 7
h(n) = 2
f(n) = 9
Neighbours:
E -> D
Cost = 2
g = 9
h = 3
f = 12

E -> G
Cost = 2
g = 9
h = 0
f = 9
```

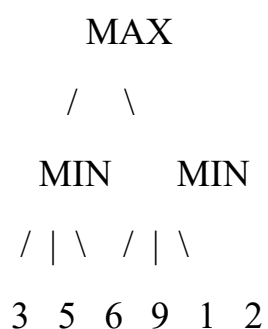
```
Output
-----
Current Node : G
Path So Far : A -> C -> E -> G
g(n) = 9
h(n) = 0
f(n) = 9
Neighbours:
-----
Current Node : D
Path So Far : A -> B -> D
g(n) = 9
h(n) = 3
f(n) = 12

Goal Reached!
Optimal Path : A -> B -> D
Total Cost : 9

=== Code Execution Successful ===
```

Question 2: Dry Run of Minimax with Alpha-Beta Pruning (10 Marks)

Consider the following game tree:



Task

Perform a **dry run of the Minimax algorithm with Alpha-Beta Pruning** by evaluating the tree **from left to right**.

For each node, show:

- Alpha (α)

- Beta (β)
- Selected Value
- Any node that gets pruned

Finally,

1. Determine the **best move for MAX**.
2. State the **final minimax value**.
3. List the **pruned nodes (if any)**.

CODE :

```
def minimax_alpha_beta(left, right):
    alpha = float('-inf')
    beta = float('inf')
    print("\n===== MINIMAX WITH ALPHA-BETA PRUNING =====")
    print("\nEvaluating Left MIN")
    left_min = float('inf')
    for value in left:
        print(f"\nVisited: {value}")
        left_min = min(left_min, value)
        beta = min(beta, left_min)
        print(f"Alpha = {alpha}")
        print(f"Beta = {beta}")
        print(f"Current MIN = {left_min}")
    print("\nLeft MIN returns:", left_min)
    alpha = max(alpha, left_min)
    beta = float('inf')
```

```

print("\nEvaluating Right MIN")
right_min = float('inf')
for value in right:
    print(f"\nVisited: {value}")
    right_min = min(right_min, value)
    beta = min(beta, right_min)
    print(f"Alpha = {alpha}")
    print(f"Beta = {beta}")
    print(f"Current MIN = {right_min}")
    if beta <= alpha:
        print("\nAlpha-Beta Pruning Occurred!")
        print("Remaining nodes are pruned.")
        break

print("\nRight MIN returns:", right_min)
final = max(left_min, right_min)
print("\n===== FINAL RESULT =====")
print("Left MIN Value :", left_min)
print("Right MIN Value:", right_min)
print("Best Move for MAX :", "Left" if left_min >= right_min else "Right")
print("Final Minimax Value :", final)

print("Enter 3 values for Left MIN")
left = list(map(int, input().split()))
print("Enter 3 values for Right MIN")
right = list(map(int, input().split()))
minimax_alpha_beta(left, right)

```

OUTPUT :

```
Output
Enter 3 values for Left MIN
3 5 6
Enter 3 values for Right MIN
9 1 2

===== MINIMAX WITH ALPHA-BETA PRUNING =====

Evaluating Left MIN

Visited: 3
Alpha = -inf
Beta = 3
Current MIN = 3

Visited: 5
Alpha = -inf
Beta = 3
Current MIN = 3
```

```
Output

Visited: 6
Alpha = -inf
Beta = 3
Current MIN = 3

Left MIN returns: 3

Evaluating Right MIN

Visited: 9
Alpha = 3
Beta = 9
Current MIN = 9
```

Output

Current MIN = 9

Visited: 1

Alpha = 3

Beta = 1

Current MIN = 1

Alpha-Beta Pruning Occurred!

Remaining nodes are pruned.

Right MIN returns: 1

===== FINAL RESULT =====

Left MIN Value : 3

Right MIN Value: 1

Best Move for MAX : Left

Final Minimax Value : 3

=== Code Execution Successful ===